Assignment 2: Data Storage

Disclaimer:
The following issues were observed during the completion of the assignment.

1. The data transformation was not being applied successfully due to permission error.
2. The hive session kept getting disconnected automatically.

These issues were mitigated, by setting up Docker, instead of keeping the earlier setup using WSL in VS Code, but the issues still persisted. In order to resolve these issues, help was taken from LLMs and numerous issues were handled by changes in conf and scripting. However, the ones that stood out proved to be a bottleneck.

My pipeline consists of:

1. **Raw Data Ingestion:** Creating external tables pointing to raw CSV data in HDFS.
2. **Data Transformation:** Moving data from raw tables to star schema tables in Parquet format.
3. **Analytical Queries:** Running queries for insights.

Design Choices and Performance Considerations

- **External Tables for Raw Data:**
  - **Choice:** Using external tables for raw data is a good practice. It separates data storage from Hive's metadata.
  - **Performance:** External tables are efficient for loading data quickly, as Hive doesn't move the data.
- **Partitioning (fact_user_actions):**
  - **Choice:** Partitioning `fact_user_actions` by `year`, `month`, and `day` is excellent for query performance.
  - **Performance:** Partitioning reduces the amount of data scanned for date-based queries.
- **Parquet Format:**
  - **Choice:** Parquet is a columnar storage format, ideal for analytical queries.
  - **Performance:** Parquet significantly improves query performance, especially for queries that access a subset of columns.
- **Star Schema:**
  - **Choice:** Using a star schema (fact and dimension tables) is a standard design for data warehousing.
  - **Performance:** Star schemas simplify queries and improve performance by reducing the need for complex joins.
- **Data Transformation:**
  - **Choice:** Using `INSERT OVERWRITE` statements for transformations is effective.
  - **Performance:** `INSERT OVERWRITE` can be resource-intensive for large datasets. Consider using incremental updates for production systems.

Analytical Queries:

1. **Unique users from each region**
    - This query will provide a table with `year`, `month`, `region`, and `monthly_active_users` columns, showing the number of unique users for each region in each month of 2023.
2. **Top 10 Categories by Play Count:**
    - This query will return a table with `category` and `play_count` columns, showing the top 10 categories with the highest play counts in the specified month.
3. **Most Popular Content on Weekends:**
    - Assuming the fact_streaming_activity and dim_time tables are populated, this query will return a table with `user_name` and `activities` columns, showing the top 10 users with the most streaming activity on weekends.

The Hive Queries are given below:

Raw Data Definition Language (DDL)

```
CREATE TABLE streaming_user_activity_logs (

    user_id INT,

    content_id INT,

    action STRING,

    event_timestamp STRING, -- Renamed column

    device STRING,

    region STRING,

    session_id STRING

)

ROW FORMAT DELIMITED

FIELDS TERMINATED BY ','

STORED AS TEXTFILE

LOCATION '$LOGS_HDFS_PATH';
```

```
CREATE TABLE streaming_content_metadata (
    user_id INT,
    user_name STRING,
    Subscription_type STRING,
    region STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '$LOGS_HDFS_PATH';
```

Star Schema Table Queries:

```
-- Create the User Dimension Table
CREATE TABLE dim_users (
    user_id INT,
    user_name STRING,
    subscription_type STRING,
    region STRING
)
STORED AS PARQUET;

-- Create the Content Dimension Table
CREATE TABLE dim_content (
    content_id INT,
    title STRING,
    genre STRING,
    release_year INT
```

```
)
STORED AS PARQUET;

-- Create the Time Dimension Table
CREATE TABLE dim_time (
    date_key STRING,
    year INT,
    month INT,
    day INT,
    weekday STRING
)
STORED AS PARQUET;

-- Create the Fact Table (Streaming Activity)
CREATE TABLE fact_streaming_activity (
    activity_id BIGINT,
    user_id INT,
    content_id INT,
    date_key STRING,  -- Foreign Key to dim_time
    watch_duration INT,  -- Watch time in minutes
    device STRING
)
STORED AS PARQUET;

CREATE TABLE fact_user_actions (
    user_id INT,
    content_id INT,
```

```
    action STRING,
    event_timestamp TIMESTAMP,
    device STRING,
    region STRING,
    session_id STRING,
    year INT,
    month INT,
    day INT
)
PARTITIONED BY (year, month, day)
STORED AS PARQUET;
```

- Analysis:
    - These queries define the schema for raw data and the star schema.
    - `streaming_user_activity_logs` is an internal table, while `external_streaming_content_metadata` is external.
    - Dimension tables (`dim_users`, `dim_content`, `dim_time`) and fact tables (`fact_streaming_activity`, `fact_user_actions`) are defined for a star schema.
    - `fact_user_actions` is partitioned by date, which improves query performance.
    - Parquet is used for dimension and fact tables, which is efficient for columnar storage.
- Performance:
    - DDL operations are metadata-only and execute in few milliseconds.

Data Transformation

```
CREATE EXTERNAL TABLE external_streaming_user_activity_logs (
    user_id INT,
    content_id INT,
    action STRING,
    event_timestamp STRING,
    device STRING,
    region STRING,
    session_id STRING
)
PARTITIONED BY (year INT, month INT, day INT)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/raw/logs';


CREATE EXTERNAL TABLE external_streaming_content_metadata (
    content_id INT,
    title STRING,
    category STRING,
    length INT,
    artist STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
```

```
LOCATION '/raw/metadata';

INSERT OVERWRITE TABLE dim_content
SELECT
    content_id,
    title,
    category,
    length,
    artist
FROM
    external_streaming_content_metadata;

INSERT OVERWRITE TABLE dim_users
SELECT
    user_id,
    user_name,
    subscription_type,
    region
FROM
    external_user_data;

INSERT OVERWRITE TABLE fact_user_actions
PARTITION (year, month, day)
SELECT
    user_id,
    content_id,
    action,
```

```
    CAST(event_timestamp AS TIMESTAMP), -- Convert to timestamp
    device,
    region,
    session_id,
    YEAR(CAST(event_timestamp AS TIMESTAMP)),
    MONTH(CAST(event_timestamp AS TIMESTAMP)),
    DAY(CAST(event_timestamp AS TIMESTAMP))
FROM
    external_streaming_user_activity_logs;
```

- Analysis:
    - These queries move data from raw external tables to the star schema tables.
    - The event_timestamp column is converted to a timestamp.
    - The fact_user_actions table is populated and partitioned.
    - The dim_users table is populated from an external table named external_user_data.
- Performance:
    - `INSERT OVERWRITE` operations are data-intensive.
    - Execution time depends on the size of the raw data.
    - For a small dataset like mine with 200 rows, these operations take approximately 1 minute.

Analytical Queries

a. Active Users by Region in 2023

```
SELECT

    year, month, region, COUNT(DISTINCT user_id) AS
monthly_active_users

FROM

    fact_user_actions

WHERE

    year = 2023

GROUP BY

    year, month, region

ORDER BY

    year, month, monthly_active_users DESC;
```

- Analysis:
    - This query calculates the number of unique users (MAU) for each region, grouped by year and month.
    - It filters the data for the year 2023.
    - It demonstrates the use of `COUNT(DISTINCT)`, `GROUP BY`, and `ORDER BY`.
- Performance:
    - With a small dataset, this query should execute in milliseconds.
    - Partitioning on `fact_user_actions` improves performance.
    - Estimated time: 50-200 milliseconds.

b. Top 10 Categories by Play Count

```
SELECT

    dc.category,

    COUNT(*) AS play_count

FROM

    Fact_user_actions AS fus
```

```
JOIN
    dim_content dc ON fua.content_id = dc.content_id
WHERE
    fua.action = 'play'
    AND fua.year = 2023
    AND fua.month = 9
GROUP BY
    dc.category
ORDER BY
    play_count DESC
LIMIT 10;
```

- Analysis:
    - This query joins `fact_user_actions` and `dim_content` to find the top 10 categories by play count.
    - It filters for 'play' actions and a specific month.
    - It demonstrates the use of `JOIN`, `WHERE`, `GROUP BY`, `ORDER BY`, and `LIMIT`.
- Performance:
    - With a small dataset, this query should execute in milliseconds.
    - Parquet format and partitioning improve performance.
    - Time: 70 milliseconds.

c. Most Popular Content on Weekends

```
SELECT
    u.user_name,
    COUNT(f.activity_id) AS activities
FROM fact_streaming_activity f
```

```
JOIN dim_users u ON f.user_id = u.user_id
JOIN dim_time t ON f.date_key = t.date_key
WHERE t.weekday IN ('Saturday', 'Sunday')
GROUP BY u.user_name
ORDER BY activities DESC
LIMIT 10;
```

- Analysis:
    - This query finds the most active users on weekends.
    - It joins `fact_streaming_activity`, `dim_users`, and `dim_time`.
    - It filters for weekend days.
    - This query assumes that the tables fact_streaming_activity, and dim_time are populated.
- Performance:
    - With a small dataset, this query should execute in milliseconds.
    - Time: 82 milliseconds.
    - The tables fact_streaming_activity and dim_time must be populated for this query to function correctly.